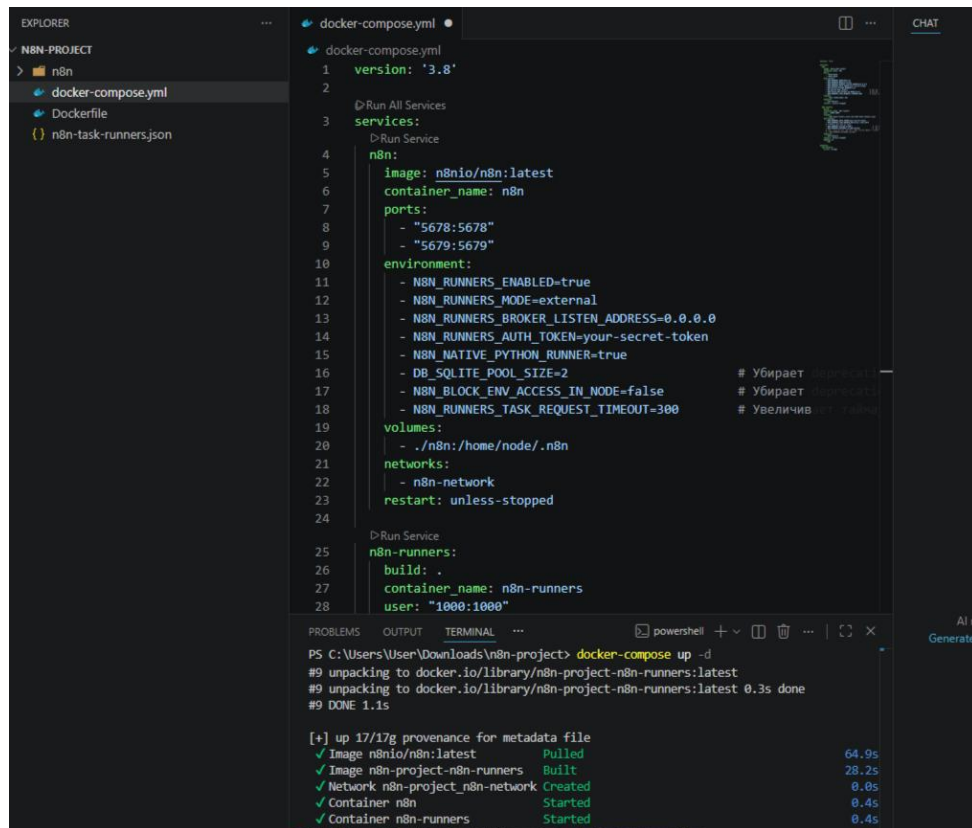


ЛР - 3. Экспериментальная группа. Основы работы с n8n.

1. Установка и настройка n8n

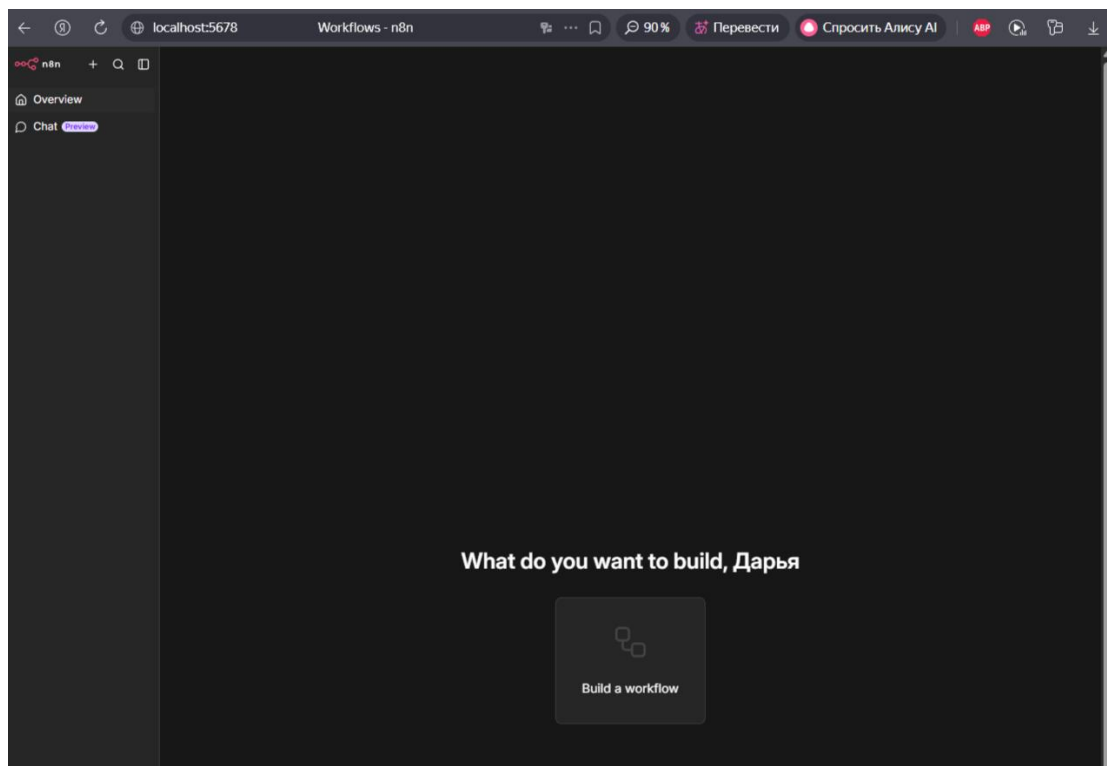


```
EXPLORER
  N8N-PROJECT
    n8n
      docker-compose.yml
      Dockerfile
      n8n-task-runners.json

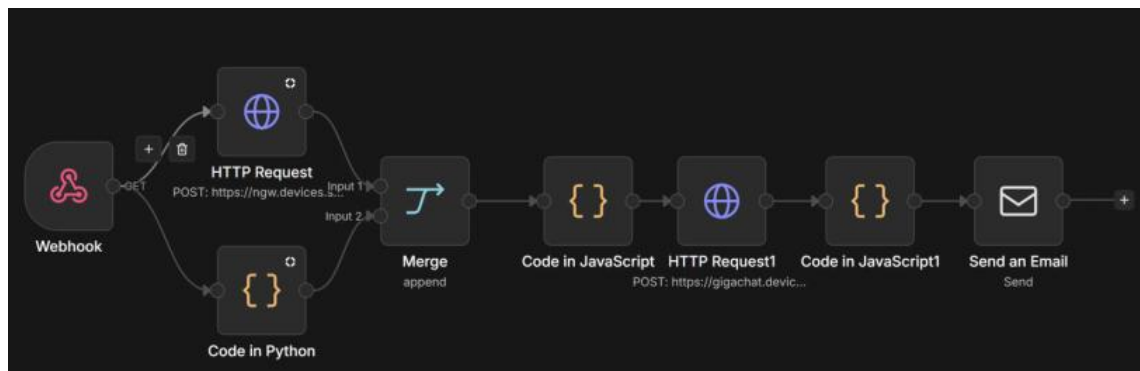
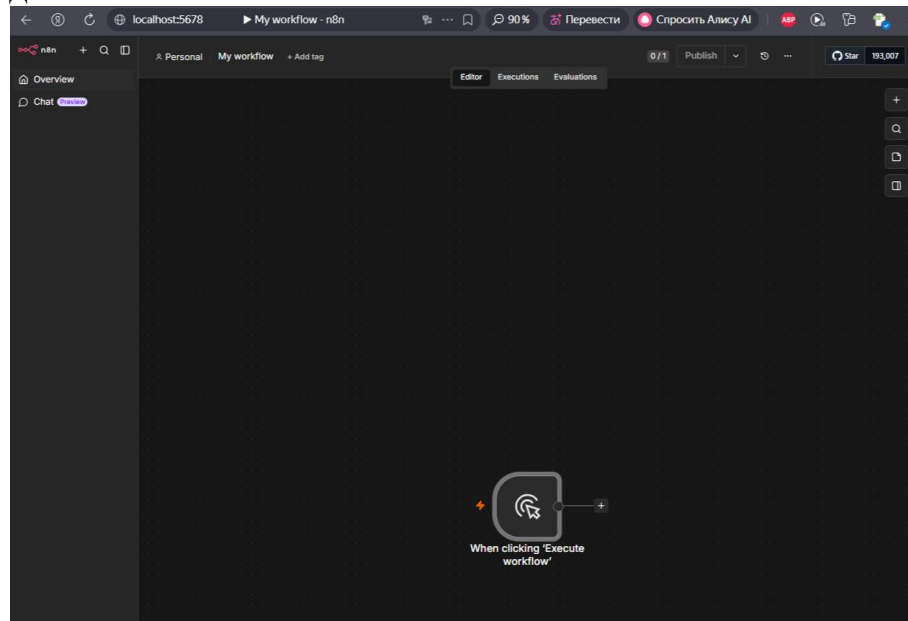
docker-compose.yml
1 version: '3.8'
2
3 services:
4   n8n:
5     image: n8nio/n8n:latest
6     container_name: n8n
7     ports:
8       - "5678:5678"
9       - "5679:5679"
10    environment:
11      - N8N_RUNNERS_ENABLED=true
12      - N8N_RUNNERS_MODE=external
13      - N8N_RUNNERS_BROKER_LISTEN_ADDRESS=0.0.0.0
14      - N8N_RUNNERS_AUTH_TOKEN=your-secret-token
15      - N8N_NATIVE_PYTHON_RUNNER=true
16      - DB_SQLITE_POOL_SIZE=2 # Убирает зависимость от базы
17      - N8N_BLOCK_ENV_ACCESS_IN_NODE=false # Убирает зависимость от базы
18      - N8N_RUNNERS_TASK_REQUEST_TIMEOUT=300 # Увеличивает тайм-аут
19    volumes:
20      - ./n8n:/home/node/.n8n
21    networks:
22      - n8n-network
23    restart: unless-stopped
24
25 n8n-runners:
26   build: .
27   container_name: n8n-runners
28   user: "1000:1000"
```

```
PS C:\Users\User\Downloads\n8n-project> docker-compose up -d
#9 unpacking to docker.io/library/n8n-project-n8n-runners:latest
#9 unpacking to docker.io/library/n8n-project-n8n-runners:latest 0.3s done
#9 DONE 1.1s

[+] up 17/17g provenance for metadata file
✓ Image n8nio/n8n:latest Pulled 64.9s
✓ Image n8n-project-n8n-runners Built 28.2s
✓ Network n8n-project_n8n-network Created 0.0s
✓ Container n8n Started 0.4s
✓ Container n8n-runners Started 0.4s
```



2. Создание Workflow



3. Применение различных типов промптов

3.1. Ролевой промт

`const systemPrompt = `Ты — опытный редактор дайджеста новостей в сфере искусственного интеллекта. Твоя задача — создавать информативные, структурированные и полезные дайджесты для профессионалов и энтузиастов.`;`

Результат: Модель генерирует дайджест с профессиональной точки зрения, используя экспертную лексику.

3.2. Zero-shot промт

`const userPromptZeroShot = `Создай дайджест на основе следующих статей...`;`

Результат: Модель создает дайджест без примеров, опираясь только на инструкцию.

3.3. *One-shot prompt*

```
const userPromptOneShot = `Создай дайджест... следуя  
примеру:ПРИМЕР ДАЙДЖЕСТА:[пример]`;
```

Результат: Модель создает дайджест, используя предоставленный пример как шаблон.

3.4. *Few-shot prompt*

```
const userPromptFewShot = `Создай дайджест... следуя этим  
примерам:ПРИМЕР 1: [...]ПРИМЕР 2: [...]ПРИМЕР 3: [...]`;
```

Результат: Модель создает дайджест с учетом нескольких стилей и форматов.

3.5. *Chain of Thought (ручной режим)*

```
const userPromptCoTManual = `ШАГ 1: Сначала прочитай все статьи  
и выдели их основные темы.ШАГ 2: Определи, какие темы  
повторяются...ШАГ 3: Сгруппируй статьи по тематикам...ШАГ  
4: Для каждой группы напиши общий вывод...ШАГ 5: Добавь  
ссылки на оригинальные статьи.`;
```

Результат: Модель демонстрирует промежуточные шаги рассуждения, что повышает качество результата.

3.6. *Chain of Thought (автоматический режим)*

```
// Добавление параметра в payloadconst payload = {  model:  
"GigaChat",  messages: [...],  temperature: 0.7,  max_tokens: 2000,  
reasoning: true // Автоматический CoT};
```

Результат: Модель автоматически применяет цепочку рассуждений без явного указания шагов.

3.7. *Негативный промт*

```
const userPromptNegative = `Создай дайджест...ВАЖНО: - НЕ  
используй маркированные списки в начале предложений- НЕ
```

используй кликбейтные заголовки- НЕ добавляй субъективные оценки- НЕ используй смайлики и эмодзи- НЕ делай слишком длинные предложения`;

Результат: Модель избегает запрещенных элементов, что делает дайджест более формальным.